Subdivision Shading for Loop Subdivision

TOM EIJKELENKAMP AND LONNEKE PULLES

University of Groningen

Compiled April 29, 2025

The combination of Phong shading with Loop subdivision results in artifacts on irregular vertices. The subdivision shading algorithm by Alexa and Boubekeur [1] solves this problem by using a slightly adapted subdivision scheme for normals in the mesh. In addition to geometric subdivision, a subdivision mask is applied to normals. The resulting normals at irregular vertices are C^1 continuous on the limit surface. In this report we explain the method and show the results of our own implementation.

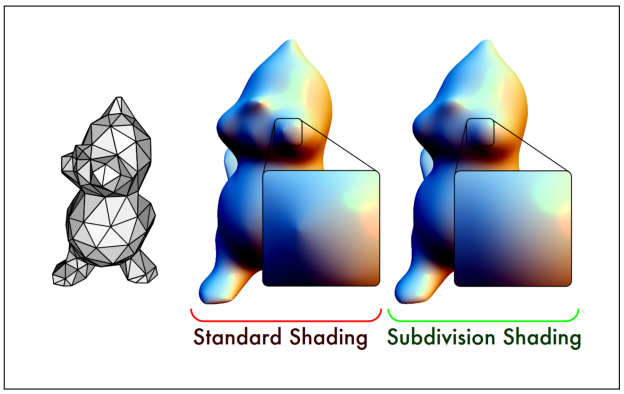


Fig. 1. The coarse TWEETY mesh is subdivided using Loop subdivision. Subdivision Shading uses subdivision rules for computing vertex normals instead of using surface normals from the limit surface. It results in visually smoother shading, especially around irregular vertices. Adapted from [1].

1. INTRODUCTION

When rendering 3D graphics on a computer, there are several properties important to look at as we talk about polygon meshes. Smooth surfaces are desired in many cases for rendering realistic objects. In mathematical terms, this means surfaces should be C^1-continuous or higher. Non-uniform rational basis splines (NURBS) were initially the most popular method, but in particular cases, the meshes had gaps between patches, thereby breaking C^1-continuity. As an alternative, subdivision become more popular, specifically for animations [3].

In this report we look into a problem that arises when using subdivision. We specifically focus on the Loop subdivision scheme. This algorithm is able to form a limit surface with C^2-continuity for regular vertices, but only C^1 continuity for irregular vertices. The tangent of the limit surface for these irregular vertices is C^0 continuous. This creates artifacts for Phong shading, which uses normals (orthogonal to the tangent plane) to compute the light reflectance. We discuss the paper [1] that poses subdivision shading as a solution to this problem. The effect of their solution to such an artifact is displayed in Figure 1.

First we will give a short summary of Loop subdivision and dicuss Phong shading in Section 2. Then we will show the solution that was given by Alexa and Boubekeur [1] in Section 4, our implementation in Section 5 and the results in Section 6. We analyze both the basic simplified algorithm and the full extended algorithm that is mathematically correct. This is followed by an implementation where regular shading is mixed with subdivision shading. Finally, in Sections 7 and 8 we draw conclusions and present possible directions of future research.

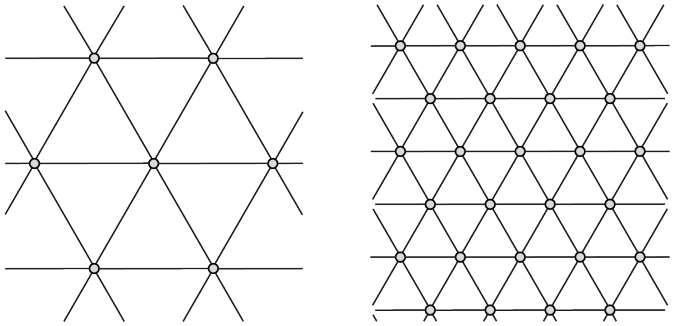
2. RELATED WORK

A. Loop subdivision

The particular subdivision algorithm we will focus on in this report was invented by Loop [5], which specifically applies to triangular meshes. Loop subdivision is based on quartic box splines, which get extended into three directions to be applied to a triangular grid. In a triangular grid, regular vertices are the vertices with a valency of 6. All other vertices are considered irregular. The algorithm results in being C^2 continuous for regular vertices (valency of 6) and G^1 continuous otherwise. This continuity property holds on the limit surface, which is the resulting surface when the mesh is subdivided infinitely many times.

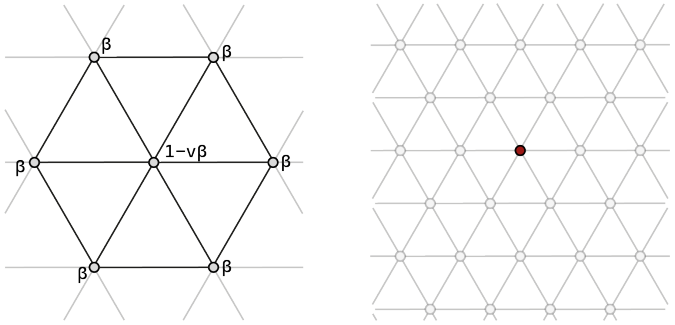
The algorithm can be seen as two consecutive steps. The first step of Loop subdivision is to do a topology refinement. Here all triangles are split into smaller triangles as shown in Figure 2.

After the new topology is computed, all the vertices are positioned according to a weighted average of their neighbors. These weights are shown in the vertex stencils displayed in Figures 3 and 4, which apply to vertices that already existed in the old topology. Figures 5b and 6b show the edge stencils, which apply to vertices that were newly created in the topology refinement.



(a) Before. **(b)** After.

Fig. 2. Topology refinement. Every triangle in the mesh is subdivided into three triangles.



(a) Vertex stencil. **(b)** Vertex to be positioned.

$$\beta = \begin{cases} \frac{3}{8v} & \text{if } v > 3 \\ \frac{3}{16} & \text{if } v = 3 \end{cases}$$

(c) Beta by Warren [6].

Fig. 3. The vertices in the more detailed topology are positioned according to a weighted average of its neighbors in the old topology. 3b shows a vertex point that exists both in the old and new topology. Its position is computed using the weights and vertex positions shown in 3a. The beta value is dependent on the valency of the vertex in question. Both for regular and irregular vertices the position can be computed using this equation.

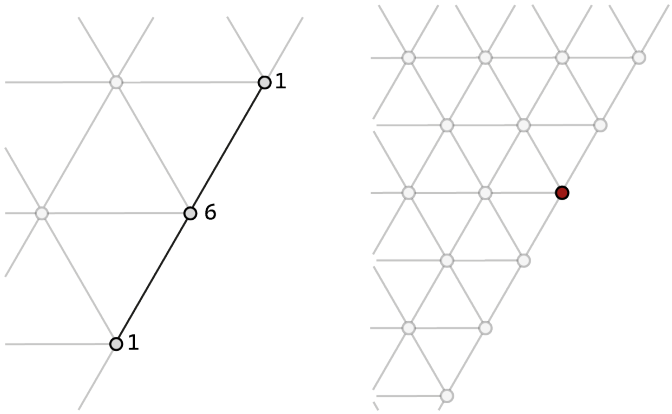
B. Phong shading

Phong shading, also named normal-vector interpolation shading, is a shading technique that, as its name implies, uses interpolation across a polygon based on vertex normals and computes pixel colors of the rasterized polygons based on the interpolated normals. This process is visualized in Figure 7.

In Phong shading, a surface point's illumination consists of three terms: ambient light, diffuse light and reflection. The formula to compute this illumination given only one light source is

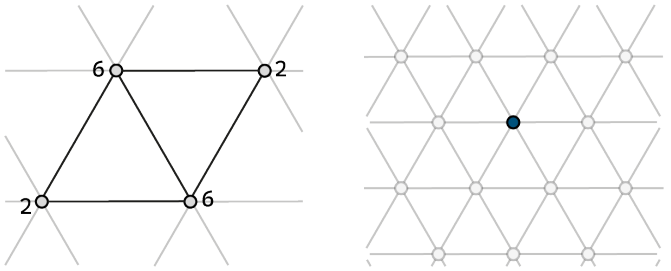
$$I_p = k_a + k_d(L \cdot N) + k_s(R \cdot V)^\alpha \quad \textbf{(1)}$$

where k_a, k_d and k_s are coefficients, L is the direction vector from the surface point to the light source, N is the surface normal, V is the direction to the camera and R is the reflection vector of the light, which are all normalized. Figure 8 visualizes these vectors.



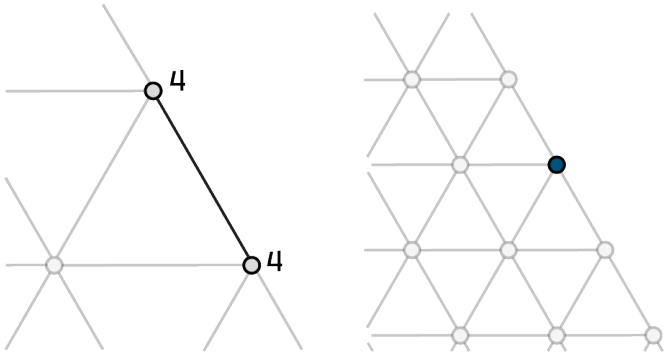
(a) Boundary vertex stencil. **(b)** Vertex to be positioned.

Fig. 4. For vertices on the boundary of a mesh a one directional uniform cubic B-spline is used. For the vertices that exist in both the old and new topology this comes down to the stencil [1 6 1].



(a) Edge stencil. **(b)** Vertex to be positioned.

Fig. 5. Vertices that were added to the mesh by the topology refinement require the stencil depicted in 5a for placement.



(a) Boundary edge stencil. **(b)** Vertex to be positioned.

Fig. 6. By topology refinement newly created vertices on the boundary use the stencil [4 4] of the uniform cubic B-spline for position computation.

3. PROBLEM DEFINITION

There occurs a problem, however, when applying Phong shading to triangular meshes. Since both the diffuse and specular term are dependent on the normals of the mesh, the quality of Phong shading depends on the continuity of these. Ideally, normals should be C^1 continuous to show smooth transitions in illumination. In the case of Loop subdivision, this generally holds except for irregular vertices, where for the limit surface

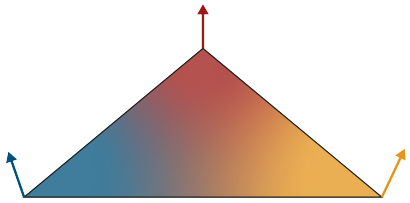


Fig. 7. Interpolation of normals, weights of respective normals displayed as colors for all positions on the triangle.

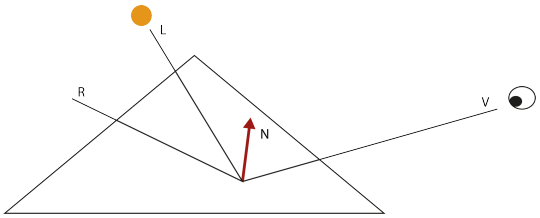


Fig. 8. Vectors involved in Phong shading.

the normals are C^0. Loop subdivision results in a mesh that is C^2 continuous in the limit, except at irregular vertices where the mesh is G^1 continuous. This means that the limit surface of a subdivided mesh's tangents and therefore its normals are generally C^1 continuous, except at irregular vertices where they are C^0 continuous. Subdivision shading tackles this problem at irregular vertices by recalculating all the normals such that they become C^1 continuous.

4. SOLUTION METHOD

A. Subdivision shading

As said, the idea of subdivision shading is to create C^1 continuity along the normals in the mesh during the shading process, including regions where there are irregular patches. Next to applying subdivision to the topology and geometry of the mesh, the normals also are recomputed using a similar subdivision operation. For every step in the subdivision algorithm all the three parts, topology, geometry and normals are computed. The subdivision-based interpolation method for normals is an alternative to the standard linear interpolation method used in Phong shading.

The algorithm for subdivision shading is as follows.

1. $N^0 = \{n_1^0, n_2^0, ...\}$ are the vertex normals of the base mesh. They could be given by the user or computed for example using an angle-weighted average of incident faces normals.

2. The subdivision of normals start out by computing for every normal a weighted average of its neighbors, where the weights are determined by the Loop subdivision mask $n^{k=0} = \sum_i w_i n_i / \parallel \sum_i w_i n_i \parallel$. This is shown in Figure 9.

3. These normals of the control vertices n_i are mapped to a plane orthogonal to the computed normal n_k. The resulting vectors are scaled according to the angle of n_i with n_k: $\tilde{n}_i = \frac{\angle(n^k, n_i)}{\|n_i - (n_i^\top n^k)n^k\|}(n_i - (n_i^\top n^k)n^k)$ This projection is visualized in Figure 10.

4. Compute a weighted average of the projected normals according to the Loop subdivision mask, i.e. $\tilde{n}^{k+1} = \sum_i w_i \tilde{n}_i$. Figure 11.

5. Map the result back onto the sphere, by rotating n^k around the axis $n_k \times \tilde{n}^{k+1}$ with an angle corresponding to $\parallel \tilde{n}^{k+1} \parallel$ to define n^{k+1}. Figure 12.

6. If $(n^k)^\top n^{k+1} > \epsilon$ go back to step 2.

Instead of using a stopping criterion with ϵ, one can also choose to implement a for-loop that runs for a concrete number of iterations to speed up computation. For our implementation described in Section 5B we used the method with a predefined number of iterations.

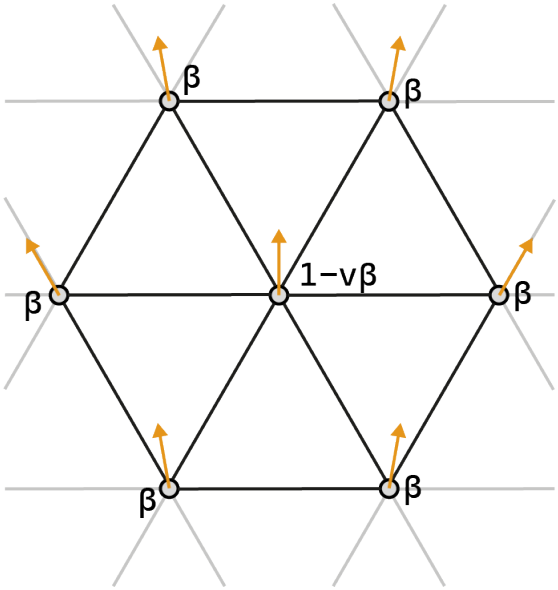


Fig. 9. Applying the loop subdivision stencil to vertex normals.

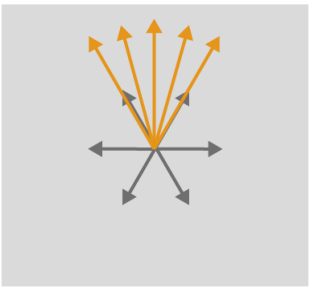


Fig. 10. The orthogonal plane projection in theory comes down to the following. Each neighboring normal is set to the same origin point. All of these normals are going to be projected on the plane orthogonal to the normal being computed. The projection is essentially the shadow that falls on the plane as an infinitely far away light source shines directly onto the plane.

The result is a normal that is averagely directed according to the subdivision mask and rotated using spherical averaging until the difference between two consecutive iterations is negligible.

B. Blending of normals

Subdivision shading improves the continuity of the normals of a subdivided mesh, enabling smoother illuminance transitions.

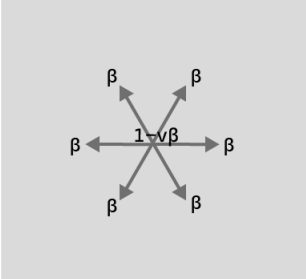


Fig. 11. Compute a weighted average of projected normals.

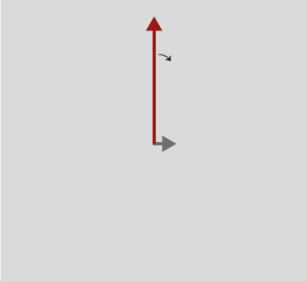


Fig. 12. Tilt normal in question on axis $n_k \times \tilde{n}^{k+1}$.

However, in some applications less smooth transitions are preferred when the contrast between regions should be highlighted. In some applications, therefore, one may want to correct the shading method only at irregular vertices. Subdivision shading is still a natural way to achieve this goal when it is extended to include the possibility to blend normals with standard Phong shading.

Each vertex is equipped with an additional *blend weight* $b_i \in [0, 1]$. Vertices obtained from subdivision are assigned blend weights based on interpolation of neighbouring vertices' blend weights. The normal that is used in Phong shading is now not only computed using the vertex normals, but it is also blended with the face normal according to the blend weight. Figure 13 shows a result of this method.

5. IMPLEMENTATION

The implementation of subdivision shading consists of three parts:

- **Linear averaging.** In each subdivision step, we implemented weighted linear averaging of the vertex normals provided by the mesh in the previous subdivision step. The weights are set by Loop's vertex and edge stencils.
- **Spherical averaging.** In addition to weighted linear averaging, an iterative spherical rotation is performed on the normal based on the angles of the neighboring normals.
- **Blending of normals.** The normals obtained from regular angle-weighted averaging of face normals of the mesh is blended with the subdivided normals according to a blend weight.

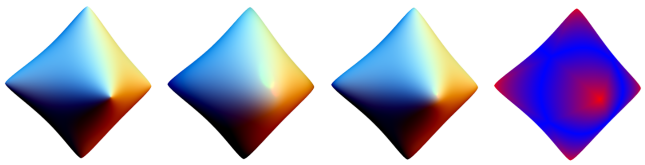


Fig. 13. Blending surface normals in the vertices with subdivided vertex normals. The images show from left to right: standard subdivision surface, subdivision shading, subdivided vertex normals blended with normals of the subdivision surface, and the color coded blend weights, where red corresponds to subdivided normals and blue corresponds to surface normals [1].

A. Linear averaging

A call to the `normalRefinement` method in the class `LoopSubdivisionShader` is added to the `subdivide` function of Loop subdivision, after calls to `geometryRefinement` and `topologyRefinement`.

In the method `normalRefinement`, each vertex normal is either subdivided with Loop's vertex stencil through a call to `vertexNormal` or Loop's edge stencil through a call to `edgeNormal` if it is a newly added vertex. Likewise, boundary vertices and edges are treated with the same weights as in Loop subdivision.

B. Spherical averaging

Spherical averaging is added as a conditional extra step after linear averaging in `normalRefinement`. It uses two functions, `sphericalAveragingVertex` and `sphericalAveragingEdge`, depending on the vertex type. These functions create exponential maps for each normal with the method `createExponentialMap`, perform weighted linear averaging of these exponential maps and accordingly rotate the normal with the function `rotateAroundAxis`. This function uses Rodrigues' rotation formula. These three steps are repeated three times. This is an alternative to using a stop criterion like in the paper, to speed up computation while yielding comparable results.

C. Blending of normals

After the added `normalRefinement` step to the `subdivide` method in `LoopSubdivider`, we now also add a `blendWeightsRefinement` step. Blend weights are not interpolated linearly during subdivision, but with weights according to Loop's vertex and edge stencils. Again, two methods contain the subdivision steps: `vertexBlendWeight` and `edgeBlendWeight`.

6. RESULTS

A. Linear averaging

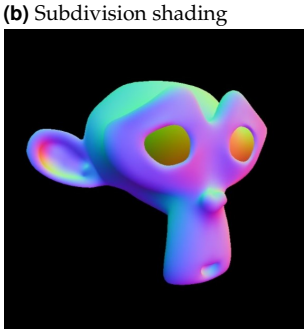
Figure 14 presents the effects of subdivision shading on a twice subdivided base mesh. We can see that details such as the creases around the eye and in the ear, and the corners of the nose and mouth get smoothed out and/or lost. Moreover, the isophotes of both the standard Phong shading method and subdivision shading show that the isophote lines are wider in areas with a higher curvature, such as the area around the eyes. On the other hand, the isophotes of abrupt creases, such as in the ear shell, seem almost cut off in Phong shading but continuous in subdivision shading.

Even though the blend weights in Figure 16j show that the upper crease of the ear only consists of regular vertices, this crease is nevertheless affected by subdivision shading. This can be explained by the fact that Loop subdivision causes C^2 continuity

(a) Phong shading

(b) Subdivision shading

(c) Standard normals



(d) Normals after subdivision

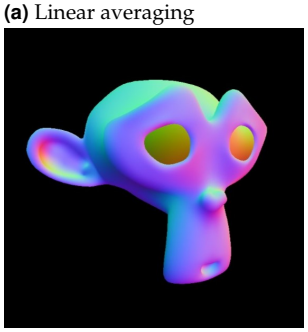
(e) Isophotes of standard normals

(f) Isophotes of subdivided normals

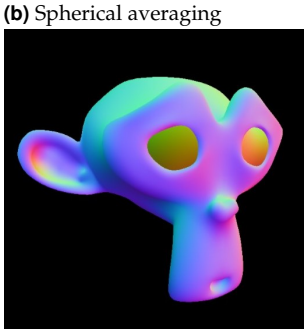
Fig. 14. Phong shading, normal shading and isophotes on regular vertex normals (angle-weighted averages of face normals) and subdivided vertex normals of the triangular mesh of Suzanne.

(a) Linear averaging

(b) Spherical averaging



(c) Normals subdivided with linear averaging



(d) Normals subdivided with spherical averaging

(e) Isophotes of normals subdivided with linear averaging

(f) Isophotes of normals subdivided with spherical averaging

Fig. 15. Phong shading, normal shading and isophotes on vertex normals subdivided with linear averaging versus spherical averaging, on the triangular mesh of Suzanne.

in regular vertices and therefore the normals of regular vertices in subdivision shading have become C^2 continuous instead of only C^1 continuous, smoothing out areas with a discontinuous curvature.

B. Spherical averaging

Figure 15 shows the difference between subdivision shading with linear averaging and with spherical averaging. As the images show, there is hardly an observable difference in Phong shading. Only when we examine the isophotes can we observe slight differences.

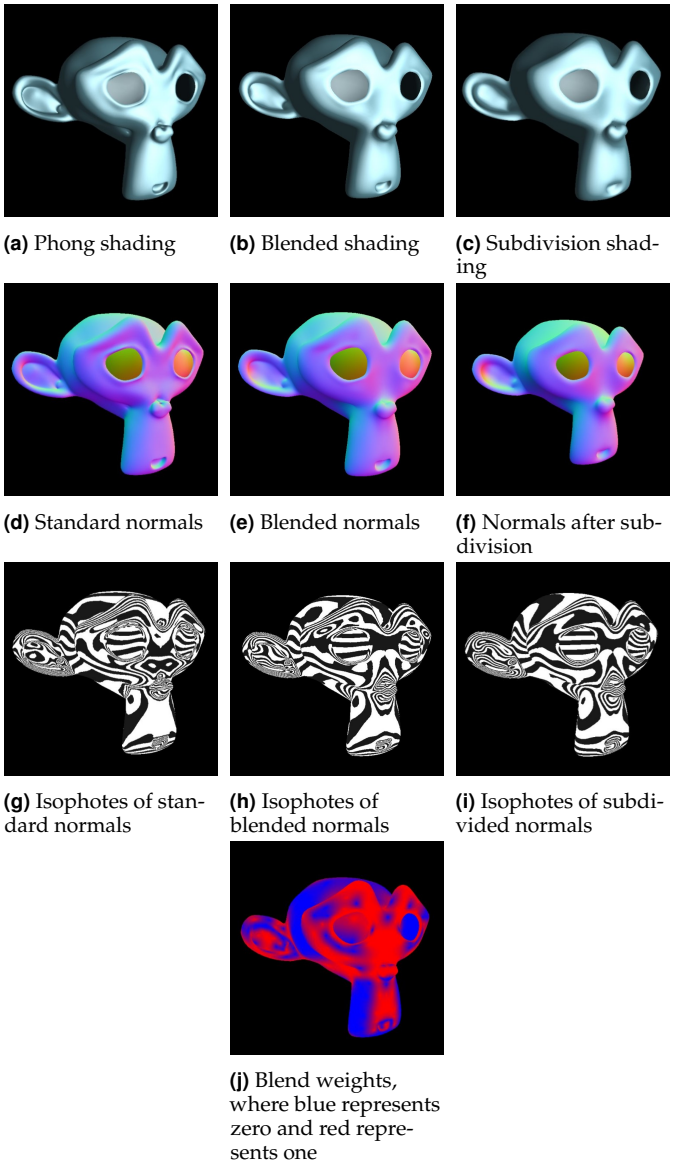
There is mathematical proof that normal refinement with spherical averaging causes C^1 continuity in the normals [2]. On the other hand, this is not the case for linear averaging. Even though there is no mathematical proof for this continuity with linear averaging, the differences between these methods seems negligible. Taking the reduced computational complexity into account, subdivision shading with linear averaging can be preferred in practical applications.

C. Blending of normals

The final results on blending the normals are depicted in Figure 16. We can see that the renderings of Suzanne with blended normals have an increased level of shading details compared to subdivision shading. This can especially be viewed in the sharp creases of Suzanne's ear and the higher curvature of the surface around the eyes.

7. CONCLUSIONS

We implemented the subdivision shading algorithm by Alexa and Boubekeur in all of its parts and showed the results of our experiments. We find that the mesh visualization indeed is improved in terms of smoothness. Only applying the linear averaging method is enough to gain this smoothness, leaving out the heavy computations on spherical averaging. The method of blending between the basic method and performing a recomputation on normals preserves the characteristics of the mesh in regular regions, but improves the visualization of irregular vertices.



(a) Phong shading **(b)** Blended shading **(c)** Subdivision shading

(d) Standard normals **(e)** Blended normals **(f)** Normals after subdivision

(g) Isophotes of standard normals **(h)** Isophotes of blended normals **(i)** Isophotes of subdivided normals

(j) Blend weights, where blue represents zero and red represents one

Fig. 16. Phong shading, normal shading and isophotes with regular, subdivided and blended vertex normals on the triangular mesh of Suzanne.

8. FUTURE WORK

Even though subdivision shading does increase the continuity of normals at irregular vertices to C^1, it also increases the continuity of the normals at regular vertices to C^2. As a result, the subdivision shading renders meshes that look too smooth for some applications and lowers the curvature in areas with many details, i.e. with high curvature. As an alternative to Loop subdivision, we could therefore also use a subdivision scheme on the normals that ascertains C^1 continuity everywhere and not C^2 continuity. An example of such a subdivision scheme is Butterfly subdivision [4]. If this alternative subdivision scheme were to be used, it is expected that shading details are conserved better than in subdivision shading. This method could therefore be an alternative to a detail-enhancing measurement like blending the normals to increase contrast around irregular vertices.

Another future direction of investigation is on the limits of subdivided normals. Our current implementation uses linear averaging or spherical averaging with three iterations. An alternative would be to directly map the normals to the limit position with the limit stencils, and to see if this improves subdivision shading for lower subdivision levels.

9. DISTRIBUTION OF WORK

We both worked on the implementation, presentation slides and the report, but we divided the work within these aspects of the final project.

Tom worked on linear averaging in the implementation, created the slides for Loop subdivision, Phong shading and spherical averaging, and wrote the introduction, related work, problem definition and solution method sections of the report.

Lonneke worked on spherical averaging, blending weights and Butterfly subdivision for normals in the implementation, created the slides for blending weights and the results, and wrote the implementation, results, conclusion and future work sections in the report.

However, we collaborated together and discussed all aspects of the project, so even the parts that we assigned to one of us, was in reality the work of both of us.

REFERENCES

1. Alexa, M. and T. Boubekeur (2008). Subdivision shading. *ACM Trans. Graph. 5*.
2. Buss, S. R. and J. P. Fillmore (2001). Spherical averages and applications to spherical splines and interpolation. *ACM Transactions on Graphics 20*, 95–126.
3. DeRose, T., M. Kass, and T. Truong (1998). Subdivision surfaces in character animation. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '98, New York, NY, USA, pp. 85–94. Association for Computing Machinery.
4. Dyn, N., D. Levine, and J. A. Gregory (1990). A butterfly subdivision scheme for surface interpolation with tension control. *ACM transactions on Graphics (TOG) 9*(2), 160–169.
5. Loop, C. (1987). Smooth subdivision surfaces based on triangles.
6. Warren, J. (1995). Subdivision methods for geometric design. *Unpublished manuscript, November*.